

Correction — TD/TP 4

Copie et déplacement de `IntArray`

MAM4 — Programmation C++

Cette correction présente une solution du parcours principal. Les tests sont à exécuter avec AddressSanitizer et UndefinedBehaviorSanitizer.

1. Diagnostic de la copie automatique

Lorsque les deux déclarations `= delete` sont retirées, le compilateur copie chaque membre :

```
b.capacity_ = a.capacity_;
b.size_ = a.size_;
b.data_ = a.data_;           // copie de l'adresse
```

Les deux pointeurs désignent donc la même allocation. `b.fill(9)` écrit dans le tableau également observé par `a`. À la sortie du bloc, les deux destructeurs exécutent `delete[]` sur la même adresse ; le sanitizer signale une double libération.

Réponse. Le compilateur ne commet pas d'erreur : il copie correctement la valeur du pointeur. C'est ce comportement membre à membre qui ne convient pas à une classe propriétaire du tableau.

2. Déclaration finale de la classe

Le fichier `int_array.hpp` contient les cinq opérations spéciales et les deux versions de `operator[]` :

```
#ifndef SEANCE4_INT_ARRAY_HPP
#define SEANCE4_INT_ARRAY_HPP

#include <cstdint>

class IntArray {
public:
    explicit IntArray(std::size_t capacity);
    ~IntArray();

    IntArray(const IntArray& other);
    IntArray& operator=(const IntArray& other);
    IntArray(IntArray&& other) noexcept;
    IntArray& operator=(IntArray&& other) noexcept;

    void push_back(int value);
    int at(std::size_t index) const;
    int& operator[](std::size_t index);
    const int& operator[](std::size_t index) const;
    int sum() const;
    void fill(int value);
    std::size_t size() const;
    std::size_t capacity() const;

private:
    std::size_t capacity_{};
    std::size_t size_{};
    int* data_{nullptr};
};

#endif
```

3. Construction et affectation par copie

Le constructeur crée une allocation différente de celle de `other`, puis copie les éléments utiles :

```
IntArray::IntArray(const IntArray& other)
: capacity_{other.capacity_},
  size_{other.size_},
  data_{capacity_ == 0 ? nullptr
        : new int[capacity_]}
{
    if (size_ > 0) {
        std::copy_n(other.data_, size_, data_);
    }
}
```

La ligne contenant `new int[capacity_]` produit la seconde allocation. L'allocation et la copie des n éléments donnent un coût en $O(n)$.

Pour l'affectation, le remplacement est préparé avant la suppression de l'ancienne ressource :

```
IntArray& IntArray::operator=(const IntArray& other)
{
    if (this == &other) {
        return *this;
    }

    int* new_data{
        other.capacity_ == 0
        ? nullptr
        : new int[other.capacity_]};

    if (other.size_ > 0) {
        std::copy_n(other.data_, other.size_, new_data);
    }

    delete[] data_;
    data_ = new_data;
    capacity_ = other.capacity_;
    size_ = other.size_;
    return *this;
}
```

Réponse. L'ordre est : allouer, copier, supprimer l'ancien tableau, installer le nouveau. Si la nouvelle allocation échoue, l'objet de gauche possède encore son ancien tableau. Le test `this == &other` traite `a = a`.

À retenir. Le constructeur de copie part d'un nouvel objet. L'affectation par copie remplace la ressource d'un objet qui existe déjà.

4. Construction et affectation par déplacement

`std::exchange` renvoie l'ancienne valeur d'un membre et la remplace par la valeur indiquée. Il permet ici de prendre les trois champs tout en vidant la source :

```
IntArray::IntArray(IntArray&& other) noexcept
: capacity_{std::exchange(other.capacity_, 0)},
  size_{std::exchange(other.size_, 0)},
  data_{std::exchange(other.data_, nullptr)}
{}
```

L'affectation doit d'abord libérer la ressource actuelle de la destination :

```
IntArray& IntArray::operator=(IntArray&& other) noexcept
{
    if (this != &other) {
        delete[] data_;
        capacity_ = std::exchange(other.capacity_, 0);
        size_ = std::exchange(other.size_, 0);
        data_ = std::exchange(other.data_, nullptr);
    }
    return *this;
}
```

Après ces opérations, la source est dans l'état valide `0 / 0 / nullptr`. Aucun élément du tableau n'a été recopié : seules deux tailles et une adresse ont été transférées. Le coût est donc en $O(1)$.

Réponse. `std::move` ne transfère pas lui-même la ressource. Il permet de sélectionner le constructeur ou l'opérateur d'affectation par déplacement ; c'est cette opération qui effectue le transfert.

5. Réutilisation d'un objet déplacé

Une capacité nulle doit pouvoir repartir à 1. La méthode complète est :

```
void IntArray::push_back(int value)
{
    if (size_ == capacity_) {
        const std::size_t new_capacity{
            capacity_ == 0 ? 1 : 2 * capacity_};
        int* new_data{new int[new_capacity]{} };

        if (size_ > 0) {
            std::copy_n(data_, size_, new_data);
        }

        delete[] data_;
        data_ = new_data;
        capacity_ = new_capacity;
    }

    data_[size_] = value;
    ++size_;
}
```

Sans le cas particulier `capacity_ == 0`, le doublement calculerait toujours une nouvelle capacité nulle.

6. Les deux versions de `operator[]`

```
int& IntArray::operator[](std::size_t index)
{
    return data_[index];
}

const int& IntArray::operator[](std::size_t index) const
{
    return data_[index];
}
```

Réponse. La première version renvoie `int&` pour permettre une écriture comme `values[0] = 42`. Sur un objet constant, la seconde version est sélectionnée et renvoie `const int&` : l'élément peut être lu, mais pas modifié par cette référence.

`operator[]` ne vérifie pas l'indice. La méthode `at()` reste l'accès vérifié.

7. Tests

Le dossier de correction contient le programme complet : copie indépendante, affectation, auto-affectation, deux déplacements, réutilisation d'une source vidée, copie d'un objet vide et accès constant. Il doit terminer par :

```
$ make run
TD4 correction: all tests passed
```

8. Règle des cinq et règle de zéro

Avec un membre `int* data_` propriétaire, nous avons dû écrire le destructeur, les deux opérations de copie et les deux opérations de déplacement. C'est la règle des cinq utilisée pour comprendre la gestion directe d'une ressource.

Avec `std::vector<int> data_`, la correspondance est :

<code>size_</code>	→	<code>data_.size()</code>
<code>capacity_</code>	→	<code>data_.capacity()</code>
<code>stockage</code>	→	géré par <code>data_</code>
<code>ajout</code>	→	<code>data_.push_back(value)</code>

Le destructeur, la copie et le déplacement générés composent alors le bon comportement de `vector`. Aucune opération spéciale n'est nécessaire : c'est la règle de zéro.

À retenir. La gestion manuelle de `IntArray` permet de comprendre la propriété d'une ressource. Dans un programme courant, on préfère laisser un type standard comme `std::vector` assurer cette gestion.

Pour aller plus loin — += puis +

```
IntArray& IntArray::operator+=(const IntArray& rhs)
{
    if (size_ != rhs.size_) {
        throw std::invalid_argument{"incompatible sizes"};
    }
    for (std::size_t i{}; i < size_; ++i) {
        data_[i] += rhs.data_[i];
    }
    return *this;
}

IntArray operator+(IntArray lhs, const IntArray& rhs)
{
    lhs += rhs;
    return lhs;
}
```

Réponse. lhs est reçu par valeur pour obtenir une copie indépendante que la fonction peut modifier. operator+ réutilise ainsi l'algorithme déjà écrit dans operator+=.